

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/298830981>

A Survey and Classification of Wireless Sensor Networks Simulators Based on the Domain of Use

Article in *Ad Hoc & Sensor Wireless Networks* · January 2014

CITATIONS

14

READS

1,325

4 authors, including:



Miodrag Zivkovic

Singidunum University

34 PUBLICATIONS 53 CITATIONS

[SEE PROFILE](#)



Bosko Nikolic

41 PUBLICATIONS 269 CITATIONS

[SEE PROFILE](#)

Some of the authors of this publication are also working on these related projects:



Cloud Computing Task Scheduling [View project](#)



Bilateral project - Slovenia - Serbia [View project](#)

A Survey And Classification Of Wireless Sensor Networks Simulators Based On The Domain Of Use

MIODRAG ŽIVKOVIĆ, University of Belgrade, School of Electrical Engineering
BOŠKO NIKOLIĆ, University of Belgrade, School of Electrical Engineering
JELICA PROTIĆ, University of Belgrade, School of Electrical Engineering
RANKO POPOVIĆ, Singidunum University, Faculty of Informatics and Computing

Wireless sensor networks (WSN) are formed by hundreds and thousands of small sensors, communicating between themselves by means of wireless communication. Due to large scale of these networks and expensive deployment, simulation is the method of choice in many projects that study and analyze the performance of a sensor network. A great number of simulators for WSN have been developed and presented in the open literature, and it is often very difficult to decide which simulator to choose as the most appropriate for different purposes. Although there are some papers that compare different WSN simulators, an exhaustive survey and classification of simulators does not exist at the moment. In this paper, we will review 20 representative simulators, most of which are open source. Then, we will identify their features, such as GUI, supported WSN models, network protocols, scalability, etc. Based on the analysis of these characteristics and the suitability for specific domains of use, simulators are classified into three major domains of use: education, research, and industrial development and design.

Reference Format:

Živković, M., Nikolić, B., Protić, J., Popović, R., 2011. A survey and classification of wireless sensor networks simulators based on the domain of use.

1. INTRODUCTION

A mobile ad-hoc network (MANET) is a self-configuring network where nodes, connected by wireless links, can move freely and the topology of network changes constantly [1]. Wireless sensor networks (WSN) can be considered as a type of mobile ad-hoc networks. These networks can be formed by hundreds and thousands of small sensors, communicating between themselves by means of wireless communication. The main reason why WSN resembles to the ad hoc network is the fact that both are distributed wireless networks, and the fact that routing between the nodes may involve the usage of intermediate relay nodes (multihop routing). Finally, self management is necessary due to the distributed nature of both, WSN and ad hoc network.

Wireless sensor networks, however, have some specific limitations [2][3]. In most cases, networks of miniature wireless sensors have to operate in long periods of time. The major limiting factor is energy consumption, both in cases that nodes have their own batteries, or when they eventually have possibility to generate energy themselves (i.e. small solar panels). In order to decrease energy consumption, most of the components of every node will be turned off for most of the time to preserve energy [4][5]. Physical world around the nodes is constantly changing, so high level of interaction between the nodes is required, in order to maintain network functionality. Nodes can also be exposed to a potentially negative environmental influence, i.e. bad weather conditions, toxic environment, radiation, etc.

All the factors mentioned above further complicate network protocols, which are necessary for communication between the nodes [6][7]. Network traffic, and even network topology also depends of dynamics of physical parameters that sensors detect. Despite all these difficulties, set up and maintenance of the nodes must remain simple and cheap. Manual configuration of large networks

consisting of small devices spread on the large area is highly impractical (almost impossible), so the nodes must have the ability to organize themselves, and to provide possibility of configuration and maintenance of the network as a whole, instead of administrating individual devices. In general, it is not feasible to test each new implementation, modification or change in configuration on real hardware, because of the tremendous effort of time and costs.

All these constraints affect the precise analytical modeling of wireless sensor networks and prediction of real performance of network protocols and communications. On the other hand, too simplified analysis leads to very questionable conclusions. Therefore, the development of test environment, which is used to analyze the behavior of network protocols and performance, is a very difficult and important task. Simulation is essential and represents the convenient method used in many projects in the domain of wireless sensor networks, and it is a common way to test new protocols and applications. This fact led to a recent expansion of simulators capable of modeling WSN. Some of the key features to be observed and evaluated are precision of the model and ability of a simulator to implement that model. Simplified hardware, protocols, and radio models lead to doubtful results and wrong conclusions. On the other hand, a realistic model, based on strong assumptions is a prerequisite to achieve credible results. However, adding too many details directly leads to very complex models, and high computational requirements are needed in order to perform all calculations. The nature of WSN, with thousands of nodes, complicates the problem further. It is very hard to find the balance between precision and level of details on the one side, and performances and scalability on the other side. Thus, implementing the simulation model is a very difficult task, which requires significant effort with a lot of tradeoff between precision and performances.

Simulator [8] is universally used to develop and test protocols of WSNs, especially in the beginning stage of these designs [9]. The cost of simulating thousands of nodes is very low, and the simulation can be finished within very short execution time. Both general and specialized simulators are available for uses to simulate WSNs. The tool, which is using firmware as well as hardware to perform the simulation, is called emulator [8]. Emulation can combine both software and hardware implementation. Emulator implements real nodes, thus it may provide more precise performance. Usually emulator has high scalability, which can emulate numerous sensor nodes at the same time. In the rest of the paper, for the cause of simplicity, we will not differentiate between simulators and emulators, and we will use only the term simulator.

In this paper, we will review 20 available WSN simulators, in order to find out how suitable they are in different areas of use. We have selected the simulators based on the information obtained from reference papers about the properties of the simulators, popularity of these tools and the available extensions for WSN and they are presented in the section 3 of this paper. We have made the reviews based on published and public available information about the simulators, as well as on individual experimentations with these tools. Summary tables with the comparisons of simulators based on the chosen set of characteristics will be presented in the sections 4 and 5 of this paper.

2. MOTIVATION AND BACKGROUND

Increasing interest in the domain of wireless sensor networks led to a real boom of available simulators. The selection of adequate simulator is a serious and time consuming task, especially in the domain of WSNs, because of complex scenarios and various protocols used in the research. Different simulators excel and underachieve in different areas, depending on project goals. The availability of a large number of simulators and specific requirements of WSN make it difficult to choose the appropriate simulator. To address this issue, we will survey some of the most popular and state of the art simulators for WSNs. Our aim is to aid educators, researchers, designers and users in the selection of an appropriate simulator for evaluation of their work. The selection of the most suitable simulator often requires the tradeoff between different characteristics: for example, some tools are designed to achieve high performance, and the others are designed to provide friendly graphical interface, etc. Our experience with development, design and evaluation of wide range of simulators and educational systems from computer architecture [10][11] to computer networks [12] and distributed systems [13][14] has shown that simulation techniques are inevitable. The

distinguishing feature of our work is that we are identifying three major domains of use of the simulators (research, education and industrial design), and we are classifying the reviewed simulators according to their usability in these three domains.

In the past three years, there have been numerous publications in the domain of wireless sensor networks. However, only a small part of these publications covers wireless sensor networks simulators, especially the conditions of their use in various domains.

Considerable research has been focused at overcoming deficiencies such as energy and computational constraints through more energy efficient routing, localization algorithms and system design. The survey [15] provides an overview of these issues and proposed solutions. Another survey paper [16] covers up open research issues and their solutions, and it also points out and depicts all important fields of sensor networks. Paper [17] analyses commercially available wireless sensor nodes, as well as research prototypes and outlines research direction in this area.

A nice review of most widely used and state of the art simulators is provided in [18], and it identifies key limitations of these tools for large scale WSNs. Simulators are compared based on a set of preferred criteria such as popularity, accessibility, complexity, etc. Several methods for simulating distributed mobile WSNs and testing the software are provided in [19]. Several papers cover the problem of modeling energy consumption of WSN. Energy consumption is one of the most important metrics in wireless sensor networks, due to limited power supply of sensor nodes, and papers [20] and [21] propose new energy models for simulation. Another important aspect of WSN is routing. The survey and classification of the state of the art routing techniques in WSN are presented in [22]. Simulation based study of ad hoc routing protocols in WSNs is presented in [23], and through analysis it shows the various intricacies that are associated with ad hoc routing protocols when used in a sensor environment. The problem of RF propagation simulation is addressed in [24], by introducing new simulation tools for radio frequency propagation and it estimates their suitability for modeling and simulation of sensor networks. Requirements for WSNs will increase in the future, and [25] estimates the suitability of five common WSN simulators for high performance sensor network planning and verification. Scalability issues are depicted in [26], together with several issues which are causing complexity of simulation. Common bases of WSN simulators are discussed in [27].

Based on our study of related papers, we have seen that most of the survey and review papers only provided a general overview of different simulators. Most of the survey papers are not exhaustive, and usually only several simulators are covered. Most of these papers cover strengths and weakness of the simulators. However, they fail to consider the limitations and suitability of these simulators in different domains of use.

In our survey, we will focus on free, open source simulators, but we will also mention some of commercial simulators for comparison. In the section 4, the overview of general purpose simulators is given: NS-2, NS-3, GNS3, Ethereal/Wireshark, OPNET, OMNeT++, GloMoSim/QualNet, J-SIM, JiST/SWANS and Ptolemy II/VisualSense. In addition to general purpose simulators, we present WSN-specific simulators, such as: TOSSIM, Castalia, EmStar, ATEMU, SENSE, SENS, Prowler/JProwler, Avrora, COOJA and Shawn.

Table I shows basic information about all described simulators, such as development team, license, web address from where the simulator can be downloaded and the used references in the literature.

Table I. Overview of simulators with basic information

Simulator	Developed by	License	Web address	References
NS2	UC Berkeley	Free	http://www.isi.edu/nsnam/ns/	[18][25][27][28][29][30][31][32][33][34]
NS3	UC Berkeley	Free	http://www.nsnam.org/	[35][36]
GNS3	EPITECH Paris	Free, with Cisco IOS	http://www.gns3.net/	[37][38][39]
Ethereal	Wireshark	Free	http://www.wireshark.org/	[40]
OPNET	OPNET Technologies	commercial, Free academic licenses	http://www.opnet.com/	[18][25][29][31][32][34][41][42]

OMNeT++	OpenSim Ltd.	Free for academic use	http://www.omnetpp.org/	[18][25][27][29][31][32][33][34][43][44]
GloMoSim/QualNet	University of California, Los Angeles	Free for academic use/commercial	http://pcl.cs.ucla.edu/projects/gloMosim/	[18][27][29][31][34][45][46]
J-SIM	University of Illinois at Urbana-Champaign	Free	http://sites.google.com/site/jsimofficial/	[27][29][31][32][34][47]
JiST/SWANS	Cornell University	Free	http://jist.ece.cornell.edu/index.html	[27][30][48]
VisualSense Ptolemy II	UC Berkeley	Free	http://ptolemy.berkeley.edu/ptolemyII/	[27][49][50][51]
TOSSIM	UC Berkeley	Free	http://www.tinyos.net/	[18][25][27][29][31][33][34][52]
Castalia	National ICT Australia	Free	http://castalia.npc.nicta.com.au/	[29][33][53]
EmStar	University of California, Los Angeles	Free	http://www.lecs.cs.ucla.edu/emstar/	[27][29][31][54]
ATEMU	University of Maryland	Free	http://www.hynet.umd.edu/research/atemu/	[27][29][31][34][55][56]
SENSE	Rensselaer Polytechnic Institute NY	Free	http://www.ita.cs.rpi.edu/sense/index.html	[18][29][31][34][57]
SENS	University of Illinois at Urbana-Champaign	Free	http://osl.cs.uiuc.edu/sens	[27][29][31][34][58][59]
JProwler	Vanderbilt University	Free	http://w3.isis.vanderbilt.edu/projects/nest/JProwler/index.html	[25][27][34][60]
Avrora	University of California, Los Angeles	Free	http://compilers.cs.ucla.edu/avrora/	[29][31][61][62][63]
COOJA	Swedish Institute of Computer Science	Free	http://www.sics.se/contiki	[33][64][65]
Shawn	Institute for Telematics, University of Lubeck, Germany	Free	http://shawn.sourceforge.net/	[29][34][66][67]

3. AVAILABLE WSN SIMULATORS

In this section we will give an overview of representative simulator environments used for WSN research, with the description of main features, key flaws and implementation issues (overview is given in Table III, at the end of this section).

Table II shows the current available version of each tool, programming language and supported operating systems on which simulator can run.

Table II. Current available version, programming language and supported platforms.

Simulator	Current version	Programming Language	Platform
NS2	ns-2.34, Jun 2009.	C++, script OTel	Windows, MAC OS, Linux, Solaris
NS3	ns-3.13, December 2011.	C++, script Python	Windows, MAC OS, Linux, Solaris
GNS3	v0.8.2, December 2011.	Python, Cisco IOS	Windows, MAC OS, Linux
Ethereal/Wireshark	1.6.5, January 2012.	C/C++	Windows, MAC OS
OPNET	Modeler 16.0, 2011.	C/C++	Windows, Linux
OMNeT++	4.2.1, January 2012.	C++	Windows, Linux

GloMoSim/QualNet	2.0, December 2000/ QualNet 5.1, 2011	Parsec	Windows, Linux
J-SIM	1.3, February 2004.	java, script jacl	Windows, Linux, MAC OS, Solaris
JiST/SWANS	v1.0.6, March 2005.	java, script Jython	Windows, Linux, MAC OS
VisualSense/Ptolemy II	8.0.1, October 2010.	Java	Windows, Linux, MAC OS, Solaris
TOSSIM	2.1.1., April 2010.	nesC, TinyOS	Windows, Linux
Castalia	3.2, March 2011.	C++	Windows, Linux
EmStar	2.5, October 2005.	Linux	Linux, Windows
ATEMU	0.4, November 2006.	nesC, TinyOS	Windows, Linux, Solaris
SENSE	SENSE, August 2004.	C++	Windows, Linux
SENS	SENS, January 2005.	C++, nesC	Linux
Prowler/ JProwler	v1.25 / v1.0, January 2004.	Java	Windows, MAC OS, Linux
Avrora	1.6.0, July 2005.	Java	Windows, MAC OS, Linux
COOJA	2.5, September 2011.	Java	Linux
Shawn	Shawn, 2004.	C++	Windows, Linux, MAC OS

3.1.NS-2

NS-2 is a discrete event simulator, developed at the Berkeley University. It is one of the most popular general purpose network simulators. NS-2 is free and open source. It is object-oriented simulator, developed in C++ programming language with OTcl as scripting support. OTcl is a scripting language with object-oriented extensions, and it is used as script interface for configuration. NS-2 implements a large amount of network protocols on all layers, waiting queues mechanisms in routers, various routing algorithms, etc. NS-2 provides most complete support of communication protocol models, comparing to all other non commercial frameworks.

The main characteristics of NS-2 are fast design and development, compatibility with various platforms, integration flexibility, the ease of use and of course the fact it is free. NS-2 is supported on Linux, MAC and Windows platforms.

NS-2 provides rich libraries of network objects and protocols. C++ hierarchy allows efficient simulation and fast execution. NS-2 is event-based simulator, and the time flow depends of events controlled by the scheduler. Every event is an object in C++ hierarchy, with unique identifier, assigned time and reference to that object. Scheduler maintains the data structure with the events needed to be executed, and triggers them one by one, by calling the event handler.

NS-2 is basically OTcl interpreter with the libraries of network objects. Language syntax is very simple, as well as integration with other languages.

NS-2 graphical support is provided via NAM, a tool for displaying the simulation results. NAM is a tool for animation, and it is used for representation of trace files of network simulation. Trace files contain information about topology, such as nodes, links , etc.

Regarding WSN support, NS-2 includes ad hoc and WSN specific protocols (i.e. directed diffusion and SMAC protocols). Several projects intended to provide WSN support to NS-2 exist, such as SensorSim and NRL. Both are extensions of NS-2 to provide WSN specific features. NS-2 can model network topologies up to one thousand nodes.

Although NS-2 has been an essential tool for network research, there has been an expansion of WSN specific simulators, because of the advantage of native sensor code simulation and the specific design of these tools. This leads to the fact that it is unclear if new WSN standards will be available for future releases of NS-2. The second problem is the object oriented design itself, which has introduced lot of unnecessary interdependencies between the modules. These interdependencies significantly complicate the addition of new protocol models, because only the NS-2 experts are able to implement them. The difficulties in extension are not a problem for traditional networks simulators, as there is a relatively small set of popular protocols. For example, for wired LAN the most widely used protocol

is Ethernet, and for wireless LAN IEEE 802.11 is the most popular protocol, etc. However, in case of wireless sensor networks, there are no such dominant protocols or algorithms. Sensor network is often tailored to the specific application, so it is highly unlikely that a single protocol or algorithm can always be the best solution for various applications.

NS-2 has been a leading simulator for network research and education. However, it has several serious flaws. Poor GUI support via NAM which only reproduces NS-2 trace files is one of them. Scalability is not implemented well enough. NS-2 is also very difficult for students. High abstraction on the network layer and the layers below leads to discontinuity between simulation and real experiments. Separated object models (OTcl and C++) are also a problem, as well as lack of documentation.

3.2.NS-3

NS-3 is a discrete event simulator. As NS-2, it was developed on Berkeley University. NS-3 was developed in C++ programming language, with optional Python scripting interface instead of OTcl. NS-3 allows scientists to analyze Internet protocols and large scale systems in a controlled environment. NS-3 is a new simulator, and it is not reverse compatible with NS-2. As NS-2, it is free, open source project, and target users are network researchers and educators.

NS-3 tries to avoid the problems NS-2 encountered, such as interoperability, lack of memory management, and complex debugging because of use of two programming languages (C++ and Tcl). NS-3 core is written in C++, with optional Python scripting, and it was designed to enhance scalability and modularity. API has a very good documentation (Doxygen), which was not a case with NS-2. Several C++ design patterns were used, such as smart pointers, templates, callback functions and copy-on-write. The possibility of aggregation of objects allows easier extension of models and packets.

Special attention was paid to realism. Internet nodes are designed as faithful representation of real objects, with support for key interfaces such as sockets and network devices, support for multiple interfaces per one node, and use of real IP addresses.

Architecture supports software integration and incorporation of different open source network software, such as kernel protocol stacks, routing daemons, packet trace analyzer and other tools (i.e. it is possible to analyze NS-3 trace files with Wireshark).

Transfer between simulation and test environment is implemented in form of virtualization. There are two modes of integration with real systems: virtual machines running on the top of NS-3 devices and channels; NS-3 stacks running in emulation mode, and sending/receiving packets through real devices.

Trace architecture in NS-3 simulator is based on a trace framework using callback design, which decouples trace sources from trace sinks, enabling customization of the tracing or statistic output without the need to rebuild simulation core.

Attribute system is also redesigned. Researchers require a possibility to identify and reassign all values used to configure parameters in the simulator. Every NS-3 object has a set of attributes: name, help text, type and initial value. System of attributes integrates handling and documentation (Doxygen) of all default and configured values: it provides control of all simulation parameters for static objects, dumping and reading them all in configuration files, and visualization in GUI.

Comparing to NS-2, NS-3 comes with a number of improvements. The most important are modular and documented core, use of C++ only (optionally Python), more faithful representation of real systems, special attention on software integration, virtualization and integration of the test environment, as well as new and up to date models.

3.3.GNS3

GNS3 is a graphical network simulator that allows simulation of complex networks. GNS3 is developed by EPITECH, Paris. It is free, open source simulator, developed in Python programming language. GNS3 allows emulation of Cisco Internetwork Operating Systems (Cisco IOS). It allows running of Cisco IOS in a virtual environment on the computer.

GNS3 is a graphical front end to a program called Dynagen. Dynamips is the core program which allows IOS emulation. Dynagen runs on top of Dynamips to create text-based, user friendly environment. GNS3 supports emulation for a long list of routers and PIX firewalls. GNS3 is a valuable tool for preparing Cisco certifications, such as CCNA and CCNP. Common router simulators are usually limited, not supporting all the commands, and only a representation of the output of a simulated router is available. GNS3 is running an actual Cisco IOS, it is possible to see exactly what is happening in IOS, and to have access to any command or parameter supported by the IOS.

Although GNS3 is a free software, due to licensing restrictions, it is required to provide own Cisco IOS to be used with GNS3. Additionally, GNS3 provides throughput of around 1000 packets per second in a virtual environment. A normal router can provide a 1000 times greater throughput. GNS3 does not take the place of a real router, but it is meant to be a tool for learning and testing in a lab environment.

3.4.Ethereal/Wireshark

Ethereal/Wireshark is free open source network simulator (traffic capture tool). It is based on the packet analysis and detailed insight in content of packets. It is developed by Wireshark Team. Wireshark is written in C/C++ programming languages. Wireshark is mostly used for network troubleshooting, testing of network security, software and communications protocol development, and as an education tool. Originally named Ethereal, in May 2006 the project was renamed Wireshark. Wireshark runs on various platforms, including UNIX and Windows.

Using Wireshark, it is possible to capture packet from the network via network interface. Wireshark also understands the structure of different network protocols, and it is able to display encapsulation and the fields of packets for a large number of network protocols. Packets are displayed with very detailed information about the protocols. For capturing packets, Wireshark uses pcap, so it is possible to capture packet on all types of networks supported by pcap. Wireshark supports import/export to a large number of other programs for analysis.

New protocols are simply added to Wireshark via plug-in, or by implementing them directly in the source code. Wireshark supports about 800 protocols at the moment, with more being added every day. Graphical interface is very simple, and it can be easily configured according to specific needs. Wireshark also has powerful filters for display and sorting of the content.

3.5.OPNET

OPNET Modeler is a discrete event network simulator, developed in C/C++ programming languages. It was developed by OPNET Technologies. OPNET is a leading simulator in the industry, specialized for network research and design. It allows design and analysis of communication networks, devices, protocols and applications, with high level of flexibility. OPNET includes graphical editor interface for creating models of different network entities, from the physical layer up to application process layer. All components are modeled using object oriented approach, which allows easy mapping to real systems, and provides flexible platform for testing of new ideas and solutions.

OPNET uses hierarchical strategy for organization of models and building the entire network. It allows simulation of complete heterogeneous network with different protocols. In the beginning, simulator was developed for military use, but eventually it evolved to a leading commercial network simulator. OPNET is very expensive for commercial use, but free licenses are available for educational purposes.

Simulator operates at the packet level. Originally it was developed for the simulation of wired networks, and it provides a large number of precise models of commercial hardware and protocols for fixed networks. It also provides wide possibilities of WSN simulations. Simulator has a lot of potential, but typically there is lack of the newest wireless systems.

OPNET is based on a very adjusted user interface, with big libraries of specific functions. Hierarchical structure divides modeling into three main domains: network domain, node domain and process domain. Network domain provides a high level description of the object contained in the system, and specifies the objects in the system, as well as their physical locations, interconnections

and configurations. Node domain specifies the internal structure of every node. Nodes can be fixed, mobile or satellite. Typical nodes are workstations, routers, satellite terminals or remote sensors. Process models are used for specification of behavior of modules which exist in the node domain. Module is modeled as a finite state machine, and it contains states with transitions and conditions between them.

OPNET supports execution of external code components. We will mention some tools of OPNET simulator: framework for editing the source code, network model editor, node model editor, process model editor, antenna model editor, link model editor, path model editor (for routing and virtual circuits), OPNET Animation viewer etc.

OPNET suffers from the same object oriented scalability problems as NS-2. It has very complex architecture which takes time to learn. The main drawback of OPNET is the fact that it is only commercially available, and the license is very expensive. Nevertheless, free academic edition of OPNET called OPNET IT Guru is available. OPNET IT Guru enables students to better understand the core concepts of networking. It allows students to simulate any network with a variety of equipment, including workstations, routers, switches, servers and links between them. It also enables users to make quick modifications to the network for latency, bandwidth and utilization, and then to study and understand the impact [68].

3.6. OMNeT++

OMNeT++ (Objective Module Network Test-bed in C++) is a discrete event modular network simulator, developed in C++ programming language. It is developed by OpenSim Ltd. It is very simple for use, due to its clean design. OMNeT++ contains strong GUI libraries for animation, tracing and debugging. The simulator also has graphical tools for simulation building and result evaluation in real time. Software is free for academic use. OMNeT++ provides a hierarchical nested architecture. The modules are programmed in C++, and the GUI is created by using the Tk library. The modules are assembled into components and models by using a high level language (NED). Modules communicate by sending messages.

Main flaw of this simulator was lack of available protocols in libraries, when compared to some other simulators. However, due to increasing popularity and extended use, new models are included in every new release. OMNeT++ simulator has very good scalability and supports very large scale network topologies, and it is limited only by available memory on the used computer.

Several protocols for localization and MAC protocols for WSN were developed using OMNeT++ simulator under Consensus project [69]. A great number of available models were developed by independent research groups, so it is sometimes difficult to combine all models.

OMNeT++ simulator mainly supports standard wired and wireless communication networks. Without the proper extensions, the simulator lacks suitable protocols and energy models for WSN, since basic support is mainly for IP networks. Today, there are several sensor network simulation frameworks based on OMNeT++. The Mobility Framework, for example, implements the support for node mobility, dynamic connection management and wireless channel model.

MiXiM [70][71], is a mixed simulator combining various simulation frameworks developed for wireless simulations in OMNeT++. It provides detailed models and protocols, with a supporting infrastructure. MiXiM combines the approaches of several existing simulators. The mobility support and general structure, for example, are taken from the Mobility Framework, the radio propagation models are taken from the Channel Simulator, and the protocol library is taken from the MAC simulator and the Mobility Framework. MiXiM introduces unique extensions such as full 3D support, models of obstacles that affect the mobility and radio signal propagation, full multi-channel support in space and frequency, etc. MiXiM scales up to 1000 nodes with low memory consumption [71].

Castalia, the most recent WSN specific simulator, completely based on OMNeT++ simulator, will be described in details later in this section.

3.7. GloMoSim/QualNet

GloMoSim (Global Mobile System Simulator) is a parallel simulation discrete event simulator for wireless and wired network systems. It was developed at UCLA, and written in Parsec programming language, simulation language based on C. Parsec provides sequential and parallel execution of simulation models based on discrete events.

GloMoSim follows the concept of OSI reference model, and uses a layered approach. Standard API is used for communication between simulation layers, enabling fast integration of new models and layers. Designers realized that using a purely object oriented approach would not be scalable. Instead, GloMoSim partitions the nodes, and each object is responsible for running one layer in the protocol stack of every node for its given partition. This approach minimizes the overhead of a large scale network. Java GUI is available for creation and configuration of different scenarios, as well as for simulation playback to confirm the results. There is a wide variety of protocols and models available in GloMoSim by default, including TCP, IEEE 802.11, MAC, UDP, HTTP, FTP, CBR, Telnet, DSR, AODV, MACA, etc. GloMoSim provides basic support for simulation of wireless and ad hoc networks (i.e. AODV, DSR). However, GloMoSim basic package does not provide support for WSN, so without additional efforts it is not possible to execute realistic simulation of WSN. GloMoSim is designed to be extensible, as all protocols are implemented as modules in the GloMoSim library.

QualNet, commercial version of GloMoSim, has been created in 2000, and GloMoSim 2.0, the last version of GloMoSim, was released under the academic license. From that moment, development of GloMoSim was stopped, and all further efforts were transferred to QualNet. The newest QualNet simulator version has a great number of improvements, including new libraries for sensor networks (ZigBee), new libraries for network security and new models for battery and energy consumption. New graphical user interface provides scenario designer, the option to observe scenarios in 2D and 3D, packet trace for debugging, and analyzer for statistic data.

3.8. J-SIM

J-SIM is an object oriented discrete event component based environment, developed entirely in Java programming language. It was developed at the Department of Computer Science at University of Illinois at Urbana-Champaign. J-SIM provides process-based real time simulations. For purpose of network modeling and simulation, a general packet switched network model is defined on the top of autonomous architecture. The basic entities of this autonomous architecture are components, which communicate with each other by sending and receiving data using their ports. The loosely coupled component architecture enables the user to design, implement and test single components individually. New components can be easily added or exchanged for existing ones. Model defines generic node structure and generic network components, which can further be used as base classes for implementation of protocols at different network layers. Although the model was developed based on common attributes of Internet, it is general enough to support other network architectures, such as WSN.

Due to its implementation in Java, J-SIM is completely platform independent, with good expansion possibilities. Packet includes scripting interface, which supports integration with a numerous scripting languages such as Perl, Tcl or Python. Actual release contains Java implementation of Tcl interpreter, called Jacl. Similar to NS-2, J-SIM is bilingual environment, where classes are written in Java, and everything is bind together with Tcl/Java. All public classes, methods and attributes are accessible directly from Jacl.

Main feature of J-SIM environment is a considerable number of supported protocols, including framework for WSN simulation with very detailed models, and implementation of localization, routing and data diffusion algorithms. J-SIM models are easy to use and change, and offer maximum flexibility. Additionally, GUI libraries for animation, trace and debug support are available. J-SIM supports networks with up to one thousand nodes. However, the project seems to be abandoned, as the last available version is from 2008.

3.9. JiST/SWANS

JiST is a high performance discrete event simulator, executing over standard Java virtual machine. It was developed at Cornell University. JiST was a pioneer of the new approach to building discrete event simulators, called virtual machine based simulation. The JiST abbreviation stands for Java in Simulation time, and the basic idea is to transform the Java virtual machine into a scheduler of events. Basically, this is done by modification of the way how methods are called between simulation entities. The resulting platform is very efficient, both in time and memory consumption [30].

Simulation environment is embedded in Java byte code. Models are implemented in Java, compiled using standard Java compiler, and run over a standard Java virtual machine. By rewriting appropriately marked Java bytecode during class loading it is possible to have registration and temporally ordered execution of simulation events. While the virtual machine itself does not know the temporally ordered execution of simulation events, JiST is implemented at a very low level which keeps the overhead for maintenance of simulation logic relatively small. The core logic of JiST is implemented by just a scheduler component, a sophisticated bytecode rewriter, marker interfaces and utility functions.

The main benefits of running inside a Java virtual machine are provided by the features of modern just in time compilation (for example profiling based optimization and aggressive inlining). One of the most important benefits is the easy integration of scripting facilities on the top of virtual machine by means of libraries like Jython, which is used as scripting support. Other important features are efficient garbage collected memory handling, cross platform support, and opportunities to benefit from future enhancements of virtual machine technologies.

The main disadvantage of JiST environment is lack of protocol models. Currently, only ad hoc network simulator SWANS (Scalable Wireless Ad Hoc Network Simulator) is supported, which has limited protocol support. SWANS is a scalable wireless network simulator, built at the top of JiST platform. It provides environment models and communication protocols for simulating MANETs. Its performance serves as validation of the virtual machine based approach to simulator construction.

SWANS is organized as independent software component, which can form complete configuration of wireless sensor networks. Using SWANS, JiST achieves high simulation throughput, saves memory, and runs standard Java network applications over simulated networks. JiST can simulate networks with up to one million of nodes, with memory consumption of about 1GB. We can see that JiST shows superior performance, which allows more detailed simulations in much shorter times, and this is possible mainly because of the very low memory consumption. Graphical interface is poor, and the only graphical aid is an event log.

3.10. Ptolemy II/VisualSense

Ptolemy II is a set of Java packets which support different simulation models (continuous time, discrete events, dataflow, rendezvous-based , etc.). It was developed at Berkeley University. The main aspect is modeling, simulation and design of concurrent, embedded real time systems. The environment supports abstract syntax, and hierarchical entities structure with ports and interconnections.

A number of simulation tools were built on the base of this set of packets, and the most interesting tool is VisualSense, projected for modeling and simulation of wireless networks. Modeling of wireless sensor networks implies sophisticated models of communication channels, sensor platforms, wireless network protocols, localization strategies, energy consumption , etc. VisualSense is a framework designed to support construction of such models, based on components. Models are developed by extending base classes or combination of available Ptolemy models. Graphical user interface is simple and intuitive, and represents graphical composition of models and results.

VisualSense supports actor-oriented definition of network nodes, wireless communication channels, physical media and wired subsystems. The architecture consists of a set of base classes for defining sensor nodes and channels, a library of subclasses that provide specific sensor and channel models, and an extensible visualization framework. It is intended to enable the researchers to share models of disjoint aspects of the sensor networks. VisualSense, however, does not provide any protocol above the wireless medium, or any sensor or physical phenomena other than sound.

3.11. TOSSIM

TOSSIM is a discrete event simulator of TinyOS [72] system. For every transferred bit an event is generated, instead of generating events at packet level. TOSSIM was developed at Berkeley University. TinyOS is an open source operating system developed for embedded wireless sensor networks. TinyOS release includes TOSSIM. TOSSIM simulates the execution of nesC code on MICA2 sensor node platform [73], and enables emulation of real hardware by mapping hardware interruptions into events. Emulated hardware components are compiled together with TinyOS components by using nesC compiler, resulting in an executable file with real TinyOS applications on the simulated physical layer. Several communication services exist which provide reading data from external sources. TOSSIM can be considered also as an emulator, as it runs actual application code. An emulator has the ability to simulate both hardware and software aspects of a model. Simulated application code can be transferred directly to the sensor platform.

TOSSIM is mainly used for simulation of TinyOS and its applications, which leads to some constraints, for example, there is no energy consumption model (however, there is an extension available for energy consumption). Another flaw is that every node must run the same code, so TOSSIM is not suitable for simulation of heterogeneous applications.

The scale of simulation is up to one thousand of nodes. Also, due to bit level simulation, performance decreases in case of larger throughputs. Each node in TOSSIM is connected in a directed graph where each edge has probabilistic bit error. Although TOSSIM monitors the behavior of TinyOS on a very low level, a number of assumptions were made in order to simplify simulation. For example, interrupts in TOSSIM are without preemption, because simulator is based on discrete events. On actual node, interrupt can happen during the execution of other code. Therefore, there is a high probability that the code running in the simulation will not run on the actual node.

There are several extensions for TOSSIM available, and we will mention the two most important. TinyViz is a visualization tool for TOSSIM, and PowerTOSSIM is an extension for energy consumption modeling.

3.12. Castalia

Castalia is a discrete event WSN simulator, developed on OMNeT++ base. As OMNeT++, it is written in C++ programming language. It was developed at National ICT Australia, under academic public license as open source project. It is a wireless sensor networks simulator, as well as simulator of general networks of embedded low power devices. Castalia can be used by researchers and developers who want to test their distributed algorithms and protocols in realistic wireless channel and radio models, with a realistic node behavior. It can also be used to evaluate different platform characteristics for specific applications, since it is highly parametric, and can simulate a wide range of platforms. It was designed for adaptation and expansion.

Castalia provides advanced channel model, based on the empirical measurement data. Channel defines a map of path loss, complex model for temporal variation of path loss, and provides full support for mobility of the nodes. Castalia also provides advanced radio model based on the real radio devices for low energy consumption communication, with probability of reception based on SINR, packet size and modulation type. Multiple TX power levels with individual node are allowed, with realistic modeling of RSSI and carrier sensing. Castalia also provides wide spectrum of sensor models. Specific characteristics were taken into account, such as clock drift effect and energy consumption of CPU module. On MAC layer, TMAC, bypass MAC and tunable MAC protocols are supported. Routing support is basic, and there are available implementations just for some basic principles, such as bypass routing, multipath rings and simple tree routing.

Sensor nodes are implemented as composite modules, containing sub modules which represent, for example, network stack layer, application or sensor. Sensor modules are further connected to wireless channel and physical process modules.

Castalia is not hardware sensor platform specific. It is not useful if user needs to test code compiled for a specific sensor platform.

3.13. EmStar

EmStar is a framework for development of WSN applications on special platforms called micro servers. These micro servers are ad hoc systems with better hardware than conventional sensor nodes. EmStar was developed at UCLA. EmStar contains Linux microkernel, libraries, services and tools. EmStar provides a flexible environment for transitioning between simulation and deployment of sensor nodes. It provides an option to interface with actual hardware while running the simulation. EmStar is a discrete event component based simulator. EmStar has several tools, and most important are EmSim, EmCee, and EmTOS.

EmSim is a simulator of the micro servers' environment. Every simulated node contains EmStar stack. Nodes are connected through simulated radio channel. Unlike other simulators which are based on discrete events, EmSim is a time driven simulator. EmCee provides an interface to real radio transmitters, instead of a simulated radio channel. EmTOS is an extension of EmStar, which allows running nesC/TinyOS applications in EmStar environment. This opens the possibility of creating heterogeneous systems of micro servers and sensors. EmTOS provides three modes of emulation: true emulation, real mode and hybrid mode. True emulation implies that all nodes are emulated by software, and real mode implies that all nodes are real devices. Hybrid mode gives an option to combine real and emulated nodes. Scale of simulation is up to 500 nodes.

EmStar's approach allows the use of the actual communication channel and sensors. EmStar code can be run on a diverse set of execution platforms, each run the same code and use the same configuration files, making it easy for developers to seamlessly iterate among all the modes of operation. Very simple environmental model and network medium can be considered as a limitation. As the purpose of the EmStar is to migrate the code to a real sensor environment, these abstractions were sufficient for the developers. Project seems to be abandoned, since there were no new developments since 2004.

3.14. ATEMU

ATEMU is an AVR processor systems emulator. It is one of the first instruction level emulators for AVR based systems. AVR processor is the type of processor used on MICA2 sensor platform [73]. It was developed at Maryland University. Together with support for AVR processors, support for other peripheral devices of MICA2 sensor platform is included, for example radio transmitter. ATEMU is used for trustworthy research of large scale sensor networks, in a controlled environment. ATEMU can also be used for educational purposes and experiments with sensor networks, without the need to buy expensive sensor hardware. ATEMU can be used directly in the development of TinyOS software, because of its compatibility with MICA2 hardware. Although current version supports only MICA2 hardware, it can simply be extended to add support for other sensor platforms. ATEMU allows simulation of several different sensors in one network.

The ATEMU framework is composed of two components, ATEMU emulator core and XATDB graphical debugger. ATEMU emulator core can simulate arbitrary number of nodes, and can model code execution on these nodes and interactions between nodes. Near complete emulation of MICA2 sensor platform is possible, with very trustworthy results. ATEMU emulator is a good platform for comparing different sensor networks protocols, and the results are more realistic comparing to other simulators. XATDB is a graphical frontend of ATEMU simulator. It provides users with a complete system for debugging and monitoring code execution. Users can execute MICA2 native code, with efficient debug support that allows setting breakpoints and step by step code execution. The powerful debugging interface can be used for multiple nodes in a sensor network at the same time.

ATEMU provides emulation of a complete hardware platform. Main advantages are options to test operating system and application, as well as simulation of heterogeneous networks of different sensor platforms. On the other hand, main flaws are high system requirements and poor scalability. Although ATEMU is the most accurate instruction level emulator for wireless sensor network, it lacks simulation speed, as it is 30 times slower than TOSSIM [63]. The project seems to be abandoned, as there was no active development since 2004.

3.15. SENSE

SENSE (Sensor Network Simulator and Emulator) is a WSN simulator, developed in C++ programming language, on top of COST, a general purpose discrete event simulator. It was developed at Rensselaer Polytechnic Institute, New York. It is designed to be an efficient, powerful and easy to use network simulator. Primary goals of SENSE simulator are expandability, reusability and scalability. Expandability is achieved by using simulation based on components with ports. SENSE model based on components with ports does not contain unnecessary interdependencies between objects, commonly present in object oriented architecture. New components can be easily added instead of old components if they have compatible interfaces, and object oriented inheriting concept is not required. Removing of unnecessary interdependencies between models also improves reusability. There is another level of reusability made possible by the use of C++ template – a component is usually declared as a template class, so it can handle different data types. Component developed for one simulation can be reused in another simulation if it meets the semantic and interface requirements. Scalability is achieved by support of parallel code execution on several computers. Parallel simulation is achieved by parallel execution of compatible components. The designers improved scalability further through the fact that all sensors use the same packet instance in memory, assuming that the packet will not be modified.

SENSE supports a large number of battery, application layer, network layer algorithms, MAC layer, physical layer and wireless channel models. It is very efficient and easy to use. Packet sharing model is an improvement in comparison to NS-2 and other object oriented models. However, SENSE is still not finished, and although the core of the simulator is stable, it still lacks models comparing to NS-2. It also lacks good visualization tool to analyze the network behavior graphically. There was no active development on this project since 2008.

3.16. SENS

SENS is a discrete event simulator, developed in C++ programming language. It was developed at Illinois University. SENS has a modular, layered architecture with components that model application running at the node, communication and physical environment. To enable realistic simulations, values from actual sensors are used to represent the behavior of implemented components. Implemented components include, among other, sound magnitude, radio signal strength characteristics and energy consumption. SENS uses simplified sensor model with three layers: application, network and physical layer, with additional combined environment and radio signal layer. A compatibility layer is provided that enables direct portability between the simulator and real sensor nodes, so that the simulator code can be used for TinyOS. NesC code can be directly used in SENS simulator. Multiple component implementations in SENS offer various levels of realism. The same source code that is executed on simulated sensor nodes in SENS can also be deployed on actual sensor nodes. SENS is platform independent – when new WSN platform appears, its parameters and profile can be easily added to the simulator. SENS introduced new mechanism for modeling of physical environment. Validity of simulation results often depends of environment model precision. User can assemble application specific environment, and such environments are modeled by different signal propagation characteristics.

SENS defines environment as grid of tiles, to provide the users with flexibility in modeling of environment and its interaction with nodes. Tiles that represent grass, concrete, walls , etc. are available, and every tile has different signal propagation characteristics. If needed, users can define their own tiles.

SENS is less customizable comparing to other simulators, as it does not provide the opportunity to change the MAC protocol, along with other low level network protocols. However, SENS use one of the most sophisticated environmental models. The only detectable phenomenon is sound and it is implemented extremely well. It can be argued whether or not it is better for a simulator to support several phenomena well, or to support one phenomenon extremely well. The project seems to be abandoned, and there was no active development since 2005.

3.17. Prowler/JProwler

Prowler is a discrete event simulator for prototyping, verifying and analyzing communication protocols for WSN. It was developed at Vanderbilt University. Prowler executes in MATLAB environment, and JProwler is built entirely in Java. Benefits gained from MATLAB environment are easy prototyping of applications, integration of different optimization algorithms. Prowler has two modes, deterministic mode and probabilistic mode. Deterministic mode can be used for application code testing, and probabilistic mode can be used for wireless communication channel and low level protocol simulations. Prowler is designed for protocol and algorithm development, optimization, parameter tuning, and it can provide QoS metrics on the performance of the tested application. However, Prowler does not provide sensor node energy models. Only one TinyOS MAC protocol has been implemented, and it also provides only two radio propagation models. The wireless channel error models are defined in only two dimensions, so it is not possible to run 3D simulations. Also, online documentation is very poor. Prowler has good extension possibilities via plug-ins.

Prowler has been used for WSN routing protocol energy cost analysis, delivery rate and packet delay calculations.

JProwler supports several radio models (Gaussian and Rayleigh), MAC protocols, and application modules. The only node model supported is MICA2 sensor node. JProwler simulator is implemented in Java and optimized for high speed. Simulation of broadcast protocol in the network of 5 000 nodes takes only few seconds in real time, however, simulation start up time during which simulator creates data structures, is very long (around 30 seconds). Graphical user interface is very basic and obscure.

3.18. Avrora

Avrora is a set of simulation and analysis tools for programs written for AVR microcontrollers. It was developed in a research project of the UCLA compiler group. Avrora is written in Java programming language, which offers great flexibility and portability. Avrora has a support for different sensor platforms, including MICA2 and MICAZ [73]. The special characteristic of Avrora simulator is that it operates on the instruction level, and the actual microcontroller programs can be executed in the simulator, instead of just simulating software models. It provides an accurate simulation of devices and radio communication, so that energy usage can be calculated according to the number of clock cycles needed for the instructions.

Avrora provides a centerpiece for research and development of sensor networks through high resolution simulation with clock cycle accurate execution of sensor nodes. In addition to emulating the execution of the program instructions, each device on each node (serial port, timers, etc) is emulated faithfully, allowing detailed analysis and verification of node behavior. Additionally, encapsulation and modularity of Avrora design allows new device simulations to be added easily, allowing exploration of new hardware sensors and the corresponding software control before the hardware is actually built. Avrora has a nearly complete implementation of the MICA2 hardware platform, including a nearly complete ATmega128L and CC1000 AM radio implementations.

The design of the Avrora has three main highlights. First, it provides a layered, well encapsulated object oriented architecture. Second, it uses the well known concept of the event queue, which keeps device operation synchronized with the main clock in an efficient manner. Third, it uses an efficient synchronization strategy that allows multiple nodes to run in parallel while preserving the timing and ordering of node interaction. Avrora runs one thread per node, and the synchronization between the threads is done only when necessary.

Avrora scales up to the network size of 10 000 nodes. Developers claim that Avrora performs as much as 20 times faster than most other simulators, with the equivalent accuracy. However, in [63], it was discovered that it is 50% slower than TOSSIM.

Avrora contains additional tools such as profiling utilities, an energy analysis tool, stack checker, control graph tool, etc. However, Avrora lacks an integrated GUI, so everything must be done manually in the command line. Second major limitation of Avrora is that it does not model clock

drift, a phenomenon where nodes may run at slightly different clock frequencies over time due to manufacturing tolerances, temperature, and battery performance. Mobility is also not supported.

3.19. COOJA

COOJA is a discrete event simulator, written in Java. It was developed by the Swedish Institute of Computer Science. COOJA network simulator is included in Contiki distribution, together with MSPSim. MSPSim can be integrated into COOJA, forming COOJA/MSPSim.

MSPSim is a firmware level simulator for sensor platforms based on Texas Instruments MSP430 microcontroller (such as Tmote Sky [74] and ESB/2 [75]). It combines cycle accurate interpretation of CPU instructions with discrete event simulation of all other components. Some components require accurate timing, while other components do not, therefore, two event queues are used, one for events based on clock cycles, and the another for events based on simulation time. MSPSim has all necessary debugging options such as break points, step by step simulation, etc.

COOJA is primarily a code level simulator for networks consisting of nodes running Contiki OS. Nodes with different simulated hardware and different software can be included in the same simulation. Code level simulation is achieved by compiling Contiki core, user processes and special simulation glue drivers into object code native to the simulator platform, and then executing this object code from COOJA.

Since COOJA is a Java application, all interaction with compiled Contiki code is done through Java Native Interface (JNI). This enables simulator to be easily extendable and allows sensor node software to be written in C programming language. Firmware level simulation can be achieved by compiling Contiki core and user processes into target platform object code that can be executed in MSPSim. COOJA is able to simulate non-Contiki nodes, which are implemented in Java, and act as a generic WSN simulator.

COOJA can simulate sensor nodes at all levels of details. Additionally, nodes simulated at different levels of details can coexist and interact in the same simulation. This feature is called cross-level simulation. COOJA combines low level simulation of sensor node hardware and simulation of high level behavior in single simulation. COOJA is flexible and extensible in that all levels of the system can be changed or replaced: sensor node platforms, OS, radio transceivers and radio transmission models. Simulator, however, due to cross level simulation and extendibility, is not very efficient. Simulation of many nodes with several interfaces each requires a lot of calculations. A test interface GUI is absent, thus making extensive and time dependent simulations difficult. Scalability is up to 10 000 nodes.

3.20. Shawn

Shawn is a discrete event simulator, written in C++ programming language. It was developed at the University of Lubeck, in Germany. Shawn has considerable differences to all other existing simulators. It supports generic high level algorithms as well as distributed protocols on exactly the same underlying protocols.

The primary design goals for Shawn are: simulate the effect caused by a phenomenon, scalability and support for extremely large networks, and free choice of the implementation model. Instead of simulating the effects of a phenomenon, Shawn simulates only the caused effects, such as, for example, for the MAC layer effect, packet lost, corruption and delay. Due to the effectiveness of this approach, this leads to a significant performance gain. Disadvantage, on the other side, is that not the same level of details as in other simulators can be reached, though a well chosen model can produce the same impacts on the application layer as detailed physical models.

To support a large amount of sensor nodes, the structure of low-level parameters is simplified so that time consuming computations can be replaced by fast substitutions, if the investigated behavior is not affected. Shawn has to be adjusted to the problem by selecting only the most important parameters for the selected scenario. Shawn supports a multiple development cycle in which developers can follow different implementation steps. Starting from an initial idea and the problem

analysis, a first centralized implementation can be created, then a simplified decentralized protocol can be developed, and finally, a fully distributed protocol.

Considering the architecture, Shawn can be divided into three parts: models, sequencer and environment. Models are the interfaces used by Shawn to control the simulation. Three basic models are the communication model, the edge model, and the transmission model. The communication model specifies whether two nodes can communicate or not. The edge model provides a graph representation of network, in which each vertex is a sensor node, and each edge is a communication link. The transmission models deals with the behavior of the transmission channel in terms of its transient characteristics. The sequencer is the central coordinating unit in Shawn. It creates the environment for the nodes, initiates the models, executes the tasks sequentially and drives the entire simulation. The simulation environment provides a virtual world in which the different parts of the simulation are located. The simulated nodes are located in a single instance of the virtual world, and the nodes are containers for processors. Application logic is implemented as instances of processors, and as a result, the application is decoupled from the node so that multiple applications can be executed in a single run.

Additional tool Viz is provided for the visualization of the simulation, which allows creation of a graphical output of a simulation. Scale of simulation is several hundreds of thousands of nodes.

3.21. Overview of simulators

The motivation to develop simulators depends on the goals and requirements imposed to developers. Every simulator fulfills the needs of the target project and users, and concentrates on specific aspects of interest. However, from the general point of view, we can identify main advantages and disadvantages of these simulators as described in Table III.

Table III. Main advantages and disadvantages of simulators

Simulator	Advantages	Disadvantages
NS2	Large community of users. Large number of available protocols.	Complex scripting. Support for only two WSN MAC protocols.
NS3	Better scalability comparing to NS2. Easier integration of external Software. Better WSN support.	Lack of some protocols available in NS2. Popularity still behind NS2.
GNS3	Design of complex topologies of high quality. Emulation of Cisco routers. Good simulation of Ethernet, ATM and Frame relay switches.	Focused on Cisco equipment. No support for WSN.
Ethereal/Wireshark	Supports hundred of protocols. Easy deep inspection. Packet sniffer – live capture and offline packet analysis.	Not a classical network simulator. Considerable protocol knowledge required for deeper analysis and inspection.
OPNET	Commercial tool. Standard for military programs. Excellent GUI.	Commercial and very expensive tool. Too complex for academic use.
OMNeT++	Very popular for education due to its expandability, and excellent online documentation. Excellent GUI. Rich model libraries.	Not sensor specific. Lack of protocols.
GloMoSim/QualNet	Parallel simulation capability. QualNet is suitable for WSN. Good GUI.	QualNet is commercial and expensive. GloMoSim is out of date, no further development after 2000.
J-SIM	Support for modeling energy consumption. Mobile WSN support. Component based architecture.	Low efficiency and speed of simulation. Only one MAC protocol supported. Project seems to be abandoned.
JiST/SWANS	Very efficient, good performance. Good scalability.	Lack of protocol models. Very poor GUI (log event only).
VisualSense/Ptolemy II	Precise and expandable radio model. Precise sound models.	No support for protocols above wireless medium. No support for other sensor and physical phenomena except for sound.
TOSSIM	Precise TinyOS simulation. Capable	MICAZ is the only supported

	for executing TinyOS application source code. Visualization tool.	mote. Preemption is not allowed. Decreased flexibility due to fact that all nodes in the network must run the same code.
Castalia	Good implementation of physical process models, sensors, clock drift, several MAC protocols and routing protocols.	Not designed only for WSN. Not applicable in need for testing code compiled for specific sensor platform.
EmStar	Support for large number of platforms. Combination of simulator and emulator. Use of component based model provides good scalability.	Code execution only for some node types. No support for parallel simulation. Not efficient and fast as some other simulators. Project seems to be abandoned.
ATEMU	Emulation on a very low hardware level of sensor node. Xatdb, debug frontend, is a good educational tool.	Original version supports only MICA2 node. Poor scalability. Slow speed of simulation. Project seems to be abandoned.
SENSE	Good tradeoff between modeling and efficiency. Fast, saves memory, easy to expand and reusable.	Not precise enough for evaluation of WSN. Lack of models. No visualization tool. Project seems to be abandoned.
SENS	User can build application specific environment. Environment is defined as a grid of tiles.	MAC protocol is not simulated precise enough. Only sensor support is for sound. Project seems to be abandoned.
JProwler	WSN simulator based on a probability. Very precise radio model.	Supports only one MAC protocol. Poor GUI.
Aurora	Very special application area, particularly for programs written for AVR microcontroller. Full support for MICA2 and MICAZ. Good scalability, up to 10 000 nodes.	It fails to model clock drift. No mobility support. Lack of visualization tool. 50% slower than TOSSIM.
COOJA	Focus on both simulated hardware and software. Large scale behavior protocols and algorithms can be observed.	Not very efficient. Supports a limited number of simultaneous node types. Extensive and time dependent simulations are difficult.
Shawn	Easy to implement distributed protocols. It can simulate vast sensor networks.	Detailed simulations of issues such as radio propagation properties or low layer issues are not well considered. No special WSN protocols, as algorithmic approach concentrates on lower layers.

4. FEATURES SPECIFIC EVALUATION

Every simulation starts with a description of the simulation model, and it contains concepts such as entities, attributes, events, etc. It is required to define the structure of the model by defining relations between the entities, their response to different events, etc.

Simulation engines and the basic models are usually available as a set of libraries in one of high level programming languages, such as C++ or Java. Also, a set of utility libraries is usually included to provide additional support, such as statistical analysis, GUI support, etc. Scripting (Tcl, for example) or mark-up language (XML, for example) are common choice to create the model description and to establish relations between entities. These scripts provide an efficient way to describe and configure the model, and also to analyze simulation results.

We can conclude that the framework for WSN simulation usually consists of basic simulation and utility libraries, and some sort of scripting support [27]. The main features that characterize good simulator [76] are support for sensor platforms and network protocols, performance, scalability,

energy consumption models, radio signal propagation models and GUI support, which will be described in the following sections.

Support for sensor platforms and network protocols

Two main aspects when evaluating simulator platforms are the support for most commonly used models, and ease of modification or integration of new models with the existing ones. The first aspect depends on the exploitation time of the simulator and the size of user community. Some early and frequently used simulators have a lot of available models, and there is high probability that some new models will be added in the new releases of these simulators. The second aspect is related to a simulator design. Good modular structure of the simulator, with well-designed interfaces enables the user to easily add new modules or modify the existing ones. Such simulators allow the users to easily create realistic simulation scenarios, and to focus on the modeling of WSN details.

Several simulators support simulation of actual sensor nodes/platforms. These specific simulators can simulate every application, protocol or component developed for the actual sensor platform. The most simulated sensor platform is MICA sensor family [73].

Typical requirements for WSNs simulators, which can be considered more or less a standard set of features, include complete TCP/IP support, support for ad-hoc routing [77][78] (i.e. AODV – Ad hoc On Demand Distance Vector), wireless MAC protocols (i.e. IEEE 802.11), radio propagation models (i.e. path loss) and mobility models [27].

Performance and scalability

In evaluation of WSN simulators, performance and scalability are the major factor [26]. Performance is usually related to the efficiency of the chosen programming language. Scalability, on the other side, depends of CPU power, available memory and data storage. Simulation of wireless sensor networks complicates these aspects further, by reducing performance and scalability, due to complexity of interaction with the environment, calculation of radio propagation, mobility and calculation of energy consumption. Assumptions which are used in the simulation of WSNs, such as idealized hardware, simplified protocols and unrealistic radio models often lead to wrong results and conclusions. For example, batteries do not supply current linearly, which affects sensitivity and transmission power. WSN simulators must have highly accurate radio models, and also new model components such as sensor node hardware, batteries, etc. However, including the high degree of detail adds high computational requirements, and leads to reduced scalability. The classical simulation tradeoff must be made, where accuracy and details are on one side, and performance and scalability on the other side. Simulation scale of several hundreds of thousands of nodes is still a challenge for existing simulators.

Energy consumption models

Nodes in WSN are usually energy-constrained and are expected to operate for long periods of time on limited energy reserves [79]. To achieve this, nodes and the embedded software must have energy-aware operation mode. The embedded software executing on the node must be aware of the state of its energy components, usually by interpreting the sampled voltage into remaining lifetime, power or energy. Based upon these sampled values, the operation mode of the node is adjusted accordingly, in order to maximize the lifetime of the network.

Effective evaluation of algorithms and protocols for energy-aware WSNs requires that the above mentioned properties are supported in simulation. Energy modeling is usually implemented via battery model and empirical values for energy consumption in different node peripherals (radio transceiver, microcontroller and sensors). Empirical consumption values are used to calculate the overall power consumption.

Radio signal propagation models

Sensor node uses omnidirectional quarter-wave monopole antenna, and there is an attenuation factor of radio signal related to the distance between transmitter and receiver [80]. The signal strength of a radio message decreases with the increase of the distance. Many authors have used propagation models to simulate algorithms for location service, routing, quality of service evaluation, etc.

The free space propagation model is largely used to simulate wireless communications. The popularity of this model comes from its simplicity [81]. However, better models must be used for better accuracy. The most frequently used models in WSN simulation are d^2 power loss, multi-path fading (two ray) model (d^4 power loss) and the ground reflection model.

GUI support

Good GUI support is important for WSN simulators, especially in the educational domain. It provides visual modeling and composition, which speeds up the design of small scale experiments and composition of basic modules. For large scale simulations, however, this is not very practical.

GUI can be considered as a debugging aid, as it allows watching the execution of simulation and enables quick detection of wrong behavior. Generally, GUI support should provide real time inspection of modules, variables and events. Also, step by step execution and availability of interaction with the simulation should be available.

GUI should also support quick visualization and inspection of results after the end of simulation.

Table IV provides overview of supported sensor platforms, WSN MAC and network protocols for each simulator.

Table IV. Supported sensor platforms, WSN MAC and network protocols

Simulator	Support for sensor platform	MAC layer	Network layer – routing algorithms
NS2	/	802.11 (several implementations), TDMA with preamble	DSDV, DSR, TORA, AODV
NS3	/	802.11(several implementations)	OLSR, AODV
GNS3	/	/	/
Ethereal/ Wireshark	/	/	/
OPNET	/	802.11, 3G, ultra Wide Band, 802.16, Bluetooth, etc.	DSR, TORA, AODV, OLSR
OMNeT++/ Castalia	/	TMAC, SMAC, adjustable MAC (approximate BMAC, LPL etc.)	Simple Tree, Multi-path rings
GloMoSim/ QualNet	/	CSMA, 802.11 and MACA	AODV, Bellman-Ford, DSR, Fisheye, LAR scheme 1, ODMRP, WRP
J-SIM	/	802.11	AODV, OSPF, multicast, shortest path tree, CBT
JiST/SWANS	/	802.11	AODV, DSR, ZRP MANET

VisualSense/ Ptolemy II	/	/	/
TOSSIM	MICAZ	Standard TinyOS 2.0 CC2420 stack	/
EmStar	support for MICA platforms	802.11, BMAC, SMAC	flooding, geographical routing, quad-tree, diffusion
ATEMU	MICA2	BMAC, SMAC	/
SENSE	/	NullMAC, 802.11	flooding, ADOV, DSR, SSR, SHR
SENS	/	standard MAC	flooding, spanning tree
JProwler	MICA2	CSMA MAC	flooding
Avrora	MICA2, MICAZ	CC1000, CC2420, 802.15.4	/
COOJA	Tmote Sky, ESB/2	X-MAC, LPP, NullMAC	/
Shawn	/	CSMA, CSMA/CA, Aloha, Random drop	/

Table V provides overview of supported characteristics of WSN simulation platform. Scalability estimations in table V were derived from documentation and specification of each simulator, and also from the test results presented in several relevant papers such as [26]. Some simulators support the energy consumption model. Simulators also differ in the implemented radio signal propagation models. Different simulators implement GUI in different ways, and we made an attempt to evaluate the GUI functionality using grades from poor to excellent.

Table V. Scalability, GUI support and supported WSN models

Simulator	Scalability	Energy consumption model	Radio signal propagation	GUI
NS2	up to 1 000 nodes	Yes	free space, two- ray ground reflection, shadowing	good
NS3	up to 20 000 nodes	Yes	free space, two- ray, shadowing, Rayleigh, Friis	good
GNS3	-	No	/	good
Ethereal/ Wireshark	-	No	/	good
OPNET	10 000 nodes	Yes	free space, two- ray, Rician fading, TIREM, etc.	excellent
OMNeT++/ Castalia	10 000 nodes	Yes	lognormal shadowing, temporal variation, unit disk	excellent

GloMoSim/ QualNet	10 000 – 100 000 nodes	Yes	free space, two ray	good
J-SIM	500 nodes	Yes	free space, two ray ground model, irregular terrain model	average
JiST/SWANS	up to 1 000 000 nodes	No	free space, two-ray ground model, Rayleigh fading, Rician fading	basic
VisualSense/ Ptolemy II	1 000 nodes	Yes	power loss channel	good
TOSSIM	1 000 nodes	yes, with Power TOSSIM	lognormal shadowing	good, with TinyViz
EmStar	500 nodes	Yes	empirical values are used for radio channel simulation.	good
ATEMU	500 nodes	No	d2 propagation	average
SENSE	1 000 nodes	Yes	free space, adjacency matrix	average, with G-SENSE
SENS	1 000 – 10 000 nodes	Yes	circular wave propagation	average
JProwler	5 000 nodes	No	Gaussian, Rayleigh	basic
Avrora	10 000 nodes	Yes	distance attenuation, indoor model	poor
COOJA	10 000 nodes	Yes	multi-path ray-tracing with support for attenuating obstacles, unit disk	good
Shawn	100 000 nodes	Yes	unit disk graph, quasi unit disk graph, radio irregularity model	average

5. USE SPECIFIC EVALUATION

We can distinguish three major domains of simulator usage: education, research and industrial design. Simulation is essential in each of these domains, but the simulator requirements differ. For example, simulator which was built for education need not to have high performance and scalability, which are desirable for simulation in other two domains. Every domain has unique set of requirements, based on the needs of target users of the simulator.

Simulators enable improvement of learning process and enhance knowledge of students by simulation of real-life situations in an easily comprehensive way. Additionally, they can be used to support university courses and laboratory exercises. Ideally, educational simulator should have some

sort of logic for knowledge evaluation of students implemented. We can identify the following requirements for educational simulators:

- Simplicity of use – the students should adapt to the new simulator quickly, and easily understand its functionality.
- Excellent GUI – Visualization is important for good learning process, as it provides insight in the real functioning of the simulated system.
- Easy to understand and follow the demonstrations – Demonstrations are used to support university courses, and they should provide the students with deeper understanding of the lecture.
- Easy to produce examples to be demonstrated – Preparation of the examples to be demonstrated should be easy enough for the teachers.
- Step by step simulation – It is essential in combination with good visualization, and it gives the students the opportunity to see step by step execution of algorithms.
- Interaction – Students should be able to interact with the simulation during its execution, for example by changing some of the parameters, and to see real time response of the system.

Scientific research in WSNs can vary from the implementation of new protocols, to the development of new routing algorithms. As we have already mentioned, simulation is by far the best solution for experimentation. Real experiments are very expensive and unpractical, if not even impossible with several thousands of nodes involved. Frequent changes to the network configuration and even more frequent runs of the experiment which are typical for research domain are only possible to be done in simulated environment. We can identify the following requirements for research domain:

- Flexibility – Frequent runs and configuration changes of the experiment require flexible simulator.
- Easy implementation of new protocols – New protocols should be easy to implement and to add to the simulator protocol base for the use in the experiments.
- Exhaustive protocol base – A large database of existing protocols should be available for experiments, and also for comparison with newly developed protocols.
- Execution efficiency – Due to generally large scale of research experiments and frequent runs, simulation execution should be efficient both in terms of speed of execution and resource consumption (available memory, etc.).
- Presentation of simulation results – Results of simulation are important in research domain, so the simulator should provide detailed metrics about the simulation run.
- Good models for energy consumption and radio signal propagation – Detailed models are required for evaluation of new algorithms and protocols.
- Scalability – As scientific research usually involves several thousands of nodes for evaluation of new algorithms, scalability is important issue.

Industrial design domain covers the commercial side of WSN simulators. It generally provides simulation tools for planning, testing and analyzing the deployment of the nodes for specific application, analyzing which node types are required, etc, before actual purchase of nodes and initial deployment. Industrial WSNs must be reliable, and while some failures may be tolerated, failure recovery is required. Power management is just as critical in the industrial environment as in other deployment scenarios. Operating and safety regulations typically call for each piece of equipment to have a dedicated power circuit, requiring separate power connections for sensors. In order to reduce costs of installation, the sensor networks must usually be battery powered. Another important requirement is security, which is necessary in all industrial applications of sensor networks [82]. Simulators in this domain are usually commercial and very expensive. The main requirements for industrial design domain are:

- Precise models of sensor nodes – Required for accurate calculations.
- Precise models of energy consumption and radio channel – Also required for accurate and precise simulation results.
- Reliable and precise simulation – Precise simulation and trustworthy results are essential, as this type of simulation is usually done before actual purchase of the

required equipment, and it could be very costly if calculations are wrong or not precise enough.

- Powerful GUI – Strong GUI support is required for professional use.
- Scalability – Although simulations are generally in smaller scale compared to research domain simulations, they can easily go to several hundreds or even thousands of nodes, so scalability is an important issue.

Classification of the simulators described in the previous chapter is not straightforward. Most of the simulators fall into more than one category. For example NS-2 is widely used in academic world as educational tool, and at the same time it is still one of the leading simulators in the research domain. The classification was based on the available information about common use of each simulator.

In the domain of education, we have placed the following simulators: NS-2, NS-3, GNS3, OPNET IT Guru, OMNeT++, Castalia and ATEMU.

Simulators that fall in the domain of research are: NS-2, NS-3, Ethereum, OMNeT++, GloMoSim, J-SIM, JiST/SWANS, VisualSense, TOSSIM, Castalia, EmStar, ATEMU, SENSE, SENS, Prowler/JProwler, Aurora, COOJA and Shawn.

There are two commercial candidates in the domain of industrial design, OPNET Modeler and QualNet. OPNET Modeler are the only commercial tools which we have reviewed. OPNET Modeler's main application is industrial and military design. QualNet is mostly used as a wireless network design tool. It must be noted here that there are several more commercial industrial simulators available, such as Simplorer (it is used for the design of mechatronic and other multi-domain systems, commonly found in the automotive, aerospace and automation industries), DynaWiz (it is used for robotic design and analysis), etc. All these simulators offer some support for designing use-specific sensor networks, but they are out of the scope of this paper and they are omitted in the survey. The classification of simulators is presented in table VI (++ means that this particular simulator is extensively used in this domain, + means that this simulator is oftenly used in this domain).

Table VI, Classification of reviewed simulators based on the common use

Simulator	Education	Research	Industrial design
NS2	+	++	+
NS3	+	++	
GNS3	++		
Ethereal/ Wireshark	+	++	
OPNET	+		++
OMNeT++	+	++	+
GloMoSim		++	
QualNet			++
J-SIM		++	
JiST/SWANS		++	

VisualSense/ Ptolemy II		++	
TOSSIM		++	
Castalia	+	++	
EmStar		++	
ATEMU	+	++	
SENSE		++	
SENS		++	
JProwler		++	
Avrora		++	
COOJA		++	
Shawn		++	

Tables VII and VIII provide comparative overview of the simulators, based on the domains of use and the criteria we have identified for each domain at the beginning of this section. These overviews give further details and information about simulators in each group. This should provide additional information to help comparing the tools in the same group and choosing the appropriate tool among several tools in the group. The overview for the domain of industrial design is presented in the table IX.

Table VII. Educational domain

Simulator	Simplicity	GUI	Step by step simulation	Interaction
NS2	low	good	yes	medium
NS3	medium	good	yes	medium
GNS3	high	good	yes	high
OPNET IT Guru	high	excellent	yes	high
OMNeT++/ Castalia	medium	excellent	yes	medium
ATEMU	medium	average	yes	high

Table VIII. Research domain

Simulator	Flexibility	Scalability	Protocol base	Presentation of results	Energy consumption model	Radio signal propagation model
NS2	high	medium	large	good	yes	good
NS3	high	high	medium	good	yes	good

Ethereal/ Wireshark	medium	-	large	good	no	-
OMNeT++/ Castalia	high	medium	medium	excellent	yes	good
GloMoSim	medium	high	medium	good	yes	average
J-SIM	medium	low	medium	average	yes	average
JiST/SWANS	medium	very high	medium	basic	no	good
VisualSense/ Ptolemy II	medium	medium	small	good	yes	good
TOSSIM	medium	medium	small	good	yes, with Power TOSSIM	basic
EmStar	medium	low	medium	good	yes	average
ATEMU	medium	low	small	average	no	basic
SENSE	medium	medium	medium	average	yes	average
SENS	medium	medium	small	average	yes	good
JProwler	medium	medium	small	basic	no	basic
Avrora	high	medium	small	basic	yes	average
COOJA	high	medium	small	good	yes	good
Shawn	medium	high	small	average	yes	average

Table IX. Industrial design domain

Simulator	Sensor models support	Scalability	Protocol base	GUI	Energy consumption model	Radio signal propagation model
OPNET	yes	high	large	excellent	yes	excellent
QualNet	yes	high	medium	excellent	yes	good
OMNeT++	yes	medium	medium	excellent	yes	good
NS2	yes	medium	large	good	yes	good

In the domain of education, we have seen that simulator should be simple to use, with a good GUI, step by step simulation and it should enable interaction with the simulation. We can note that neither one of available simulators is not ideal. Most of the simulators are very complicated for educational use, and graphical interface and editor are not adapted to students. From the descriptions and outcomes from several reference papers, we can conclude that NS-2 and OMNeT++ are most suitable for education. NS-2 is still the most popular simulator of all available simulators and common choice for academic research. The thing that keeps its popularity is a very large community of users who help each other with problems via mailing list and forums. NS-3 follows this trend, but it still hasn't achieved the popularity of NS-2. On the other hand, OMNeT++ is gaining popularity in academic world, with its well designed engine and powerful GUI. However, OMNeT++ lacks models and user base that NS-2 has. Castalia, extension of OMNeT++ gives a good basis as a teaching support in the area of WSN. GNS3 is the only typical educational simulator, but it is focused on Cisco equipment and routers, and lacks wireless support. OPNET IT Guru enables the students to create virtual network consisting of relevant hardware, protocols and application software. Once a virtual network is created, it can be manipulated in various ways – nodes can be added or subtracted, protocols can be altered, etc. The effects can then be usefully examined and analyzed. However, support for WSNs is very limited and includes only basic simulations. ATEMU can also be used as teaching tool, as it can be used in educational environment to facilitate experimentation with sensor networks. Educational field remains completely opened for development of new educational simulators.

Most of the available WSN simulators fall into the domain of scientific research. NS-2 is still the most popular simulator for researchers [83]. NS-3 provides solutions for all major NS-2 problems, such as poor scalability, and it is gaining popularity in the research community. OMNeT++ and Castalia are also very popular simulators as they were developed for research. However, OMNeT++ lacks in available protocol models compared to other simulators, such as NS-2 for example. Castalia

provides realistic models of radio channel and above average sensing models [33]. Castalia is in state of intensive development, but it has not made significant impact yet. This increases development time significantly. JiST/SWANS enables research of extremely large networks, as it supports scalability up to several hundreds of thousands nodes. GloMoSim, by means of parallel simulation, also supports high scalability for the needs of scientific research and experiments. Parallel simulations generally have better performance and scalability than sequential ones [26][27]. TOSSIM and ATEMU are also common choice for wireless sensor network research, due to emulation of TinyOS and AVR processor, respectively. EmStar allows several modes of emulation, and gives the possibility of creating hybrid networks which combine real and emulated nodes, which is significant for research. Ethereal/Wireshark is often used in research and development of the new network protocols. Avrora, with a set of simulation and analysis tools for AVR microcontrollers, provides an excellent starting point for research of sensor networks. Complete hardware emulation of MICA2 and MICAZ sensor platforms provides realistic results and allows detailed analysis of node behaviour. COOJA simulator, together with MSPSim, provides complete hardware emulation of MSP430 based sensor nodes, and enables extensive research for this platform. Shawn is good choice for development of localization algorithms and different distributed protocols. SENSE, SENS and JProWler also fall in this category. Although they do not have a large community of users, they are mostly used in scientific research. Described simulators differ in graphical user interface support. Leading simulators in this field are OMNeT++ and Ptolemy II, with powerful GUI libraries and the best graphical presentation. JiST is at the end of the list, with event log viewer only. Concerning the amount of effort it takes to become familiar with a simulator, we can observe a clear order from NS-2 at the top, over OMNeT++ and Castalia, which are relatively easy to learn, to J-SIM which is considered to be easy to learn [32]. This is probably caused by the number of supported features in NS-2, especially when regarding their configuration capabilities. When we compare performance of the simulators, general rule is that simulators developed in C/C++ programming languages have better performances than simulators written entirely in Java. Java is a memory-consuming language, and it is also slow because of an extra layer between the system and the program. This extra layer is Java Virtual Machine [84]. As for every rule, there are always exceptions, some newer Java simulators such as JiST/SWANS have better performance than C/C++ based NS-2 simulator [30]. When we talk about the available online documentation, we must distinguish OMNeT++ and Castalia, as they have well written and up to date documentation. NS-2, although very popular, has fragmented documentation. Some other simulators, such as SENS, SENSE, EmStar and ATEMU, seem to be abandoned, with no further development. Accordingly, the available documentation for these simulators is limited and not up to date.

The domain of industrial design is the most challenging, and typically these simulators are commercial and very expensive. Also, simulators in this domain are not standardized, as every commercial simulator is tailored to very specific requirements and has specific solutions (Simplorer – tailored to aerospace and automotive industries, for example). In our survey, we have included two commercial representatives of this category, OPNET and QualNet. OPNET is a leading and most popular simulator for industrial design. It has high level of flexibility in design of communication networks and devices, and it is a standard in military programs. QualNet, the commercial successor of GloMoSim, is a high-fidelity network simulator which is suitable for network design, testing and modeling. QualNet has a wide range of possible applications and sophisticated animation capabilities, with powerful GUI.

Classification described above is presented on the Table VI. As we can see, there is no simulator that is universal and complete solution for all domains. The choice of simulator must be based on the project requirements and intended domain of use.

6. CONCLUSION

In this paper, we have reviewed and investigated a set of representative currently available WSN simulators.

Our goal was to provide more exhaustive coverage of available simulators than each of previous surveys did, and to provide their comprehensive characterization. We have taken into account both

general network simulators that were extended for WSN, and, on the other hand, simulators that were explicitly created for the use in WSN domain. We have presented the total of 20 different simulators, and we have given the comparative overview of these simulators. We have identified three main domains of use: research, education and industrial design. Simulation is essential in each of these domains, although the requirements and expectations of users differ. We have defined the requirements for each domain of use, and classified the presented simulators in these three domains, using and overlapping classification..

Our overall conclusion is that, although there are several simulators even beyond the presented selection of simulators, none of the available simulators presents the universal and complete solution for all domains. The choice of simulator must be based on the project requirements and the most important aspects that the researcher wants to investigate. We hope that this paper will help users to choose an appropriate simulators for their evaluation, while taking into account the simulator comparisons provided in tables, and classification into three main domains of use. Because of the obvious differences of the simulation environments and the most important aspects that one wants to investigate, one could use our work as a starting point to investigate available simulators that are a possibility for planned use.

As we have seen in the previous section, there is a lot of room for improvements in the domain of wireless sensor network simulation for educational purposes. None of the existing solutions provide dedicated features for educational and learning objectives. The existing simulators are either too complex, or too specific, so the development of a simulator that addresses all the requirements of the educational domain would be of interest for the academic environment.

Majority of the available simulators fall in the research domain. The variety of available simulators gives researchers the possibility to choose the most adequate solution, but also to extend the features of the existing open source solutions. Future work in this area mainly would be to continuously develop and integrate new protocols, routing algorithms and sensor models.

In the domain of industrial design, there are no standards, as almost all commercial tools are tailored to specific intended use. However, specific requirements and high standards of industrial design may motivate the evolution of various simulators developed for research purposes to extended or customized versions for industrial use.

REFERENCES

1. J. Antonio Garcia-Macias, Javier Gomez-Castellanos, "MANET versus WSN" in *Sensor Networks and Configuration: Fundamentals, Standards, Platforms, and Applications*, Springer-Verlag, Germany. 2006. ISBN 3540373640.
2. Z. Hu, B. Li, "Fundamental Performance Limits of Wireless Sensor Networks", in *Ad Hoc and Sensor Networks*, Nova Science Publishers, pp. 81-101, ISBN 1-59454-396-8, Hardcover, 2005.
3. A. Deshpande, C. Guestrin, S.R. Madden, "Resource – aware Wireless Sensor – Actuator Networks", *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, 2005.
4. Y. Li, W. Ye, J. Heidemann, "Energy Efficient Network Reconfiguration for Mostly-Off Sensor networks", in *Proceedings of the Third IEEE Conference on Sensor and Adhoc Communication and Networks*, pp. 527-535. Reston, Virginia, USA, IEEE. September, 2006
5. B. Hohlt, L. Doherty, E. Brewer, "Flexible Power Scheduling for Sensor Networks", in *Proceedings of the Third International Symposium on Information processing in sensor networks*, ACM New York, NY, USA 2004, ISBN: 1-58113-846-6
6. L.J.G. Villalba, A.L.S.Orozco, A.T. Cabrera, C.J.B. Abbas, *Sensors* 2009, ISSN 1424-8220, 2009.
7. S. Bala, "Secure Routing in Wireless Sensor Networks", Master thesis, Thapar University, Patiala, 2009.
8. M. Imran, A.M. Said, H. Hasbullah, A Survey of Simulators, Emulators and Testbeds for Wireless Sensor Networks", *Information Technology(ITSim)*, 2010 International Symposium in June 2010, pp. 897 – 902, ISBN: 978-1-4244-6715-0
9. F. Yu, "A Survey of Wireless Sensor Network Simulation Tools", <http://www1.cse.wustl.edu/~jain/cse567-11/ftp/sensor/index.html>
10. B. Nikolic, Z. Radivojevic, J. Djordjevic, V. Milutinovic, "A Survey and Evaluation of Simulators Suitable for Teaching Courses in Computer Architecture and Organization", *IEEE, Transactions on Education* Vol. 52, No. 4, pp. 449-459, November 2009
11. J. Djordjevic, B. Nikolic, A. Milenkovic, "Flexible Web-based Educational System for Teaching Computer Architecture and Organization," *IEEE, Transactions on Education*, Vol. 48, No. 2, pp. 264-274, May 2005
12. N. Jovanovic, R.M. Popovic, Z. Jovanovic, "WNetSim: a web-based computer network simulator", *International Journal Of Electrical Engineering Education*, vol. 46, pp. 383-396, 2009

13. J. Djordjevic, B. Nikolic, M. Mitrovic, "A Memory System for Education," *The Computer Journal*, Vol. 48, No. 6, pp. 630-641, November 2005
14. J. Protic, M. Tomasevic, V. Milutinovic, "Distributed Shared Memory: Concepts and Systems", *IEEE Parallel & Distributed Technology: Systems & Technology*, Volume 4 Issue 2, June 1996
15. A. Bharathidasan, V.A.S. Ponduru, "Sensor Networks: An Overview", Department of Computer Science, University of California.
16. K.C. Rahman, "A Survey on Sensor Network", *JCIT*, ISSN 2078-5828 (print), ISSN 2218-5224 (online), volume 01, issue 01, 2010
17. V. Potdar, A. Sharif, E. Chang, "Wireless Sensor Networks: A Survey", *waina*, pp.636-641, 2009 International Conference on Advanced Information Networking and Applications Workshops, 2009
18. M.Z. Khan, B. Askwith, F. Bouhafs, M. Asim, "Limitations of Simulation Tools for Large-Scale Wireless Sensor Networks", *waina*, pp.820-825, 2011 IEEE Workshops of International Conference on Advanced Information Networking and Applications, 2011
19. D.J. Anthony, F. Chavez-Ramirez, M.B. Dwyer, M. C. Vuran, S. Elbaum, W.P. Bennet, "Simulating and testing mobile wireless sensor networks", *Proceedings of the 13th ACM international conference on Modeling, analysis, and simulation of wireless and mobile systems (MSWIM '10)*, pp. 49-58, October 2010.
20. W. Du, F. Mieleveille, D. Navarro, "Modeling Energy Consumption of Wireless Sensor Networks by SystemC", *icsnc*, pp.94-98, 2010 Fifth International Conference on Systems and Networks Communications, 2010.
21. P. Soreanu, Z. Volkovich, "Energy-Efficient Circular Sector Sensing Coverage Model for Wireless Sensor Networks", *sensorcomm*, pp.229-233, 2009 Third International Conference on Sensor Technologies and Applications, 2009
22. K. Pavai, A. Sivagami, D. Sridharan, "Study of Routing Protocols in Wireless Sensor Networks", *act*, pp.522-525, 2009 International Conference on Advances in Computing, Control, and Telecommunication Technologies, 2009
23. S. Gowrishankar, Subir Kumar Sarkar, T.G. Basavaraju, "Scenario Based Simulation Study of Ad Hoc Routing Protocol's Behavior in Wireless Sensor Networks", *icfcc*, pp.527-532, 2009 International Conference on Future Computer and Communication, 2009
24. M. Korkalainen, M. Sallinen, "A Survey of RF-Propagation Simulation Tools for Wireless Sensor Networks", *sensorcomm*, pp.342-347, 2010 Fourth International Conference on Sensor Technologies and Applications, 2010
25. M. Korkalainen, M. Sallinen, N. Kärkkäinen, P. Tukeva, "Survey of Wireless Sensor Networks Simulation Tools for Demanding Applications", *icns*, pp.102-106, 2009 Fifth International Conference on Networking and Services, 2009
26. E. Egea-Lopez, J. Vales-Alonso, A. Martinez-Sala, P. Pavon-Marino, J. Garcia-Haro, "Simulation Scalability Issues in Wireless Sensor Networks", *IEEE Communications Magazine*, July 2006.
27. E. Egea-López, J. Vales-Alonso, A. S. Martínez-Sala, P. Pavón-Mariño, J. García-Haro, "Simulation Tools for Wireless Sensor Networks", Department of Information Technologies and Communications, Polytechnic University of Cartagena, Spain, SPECTS 2005
28. The Network Simulator, NS-2 (online), <http://www.isi.edu/nsnam/ns>
29. A. Kellner, K. Behrends, D. Hogrefe, "Simulation Environments for Wireless Sensor Networks", Georg-August-Universität Göttingen, Institute of Computer Science, June 2010.
30. E. Schoch, M. Feiri, F. Kargl, M. Weber, "Simulation of Ad Hoc Networks: ns-2 compared to JiST/SWANS", Ulm University, Institute of Media Informatics
31. D. Curren, "A Survey of Simulation in Sensor Networks", University of Binghamton
32. M. Köksal, "A Survey of Network Simulators Supporting Wireless Networks", Middle East Technical University, Ankara, October 2008.
33. M. Jevtić, N. Zogović, G. Dimić, "Evaluation of Wireless Sensor Network Simulators", 17th Telecommunications forum TELFOR 2009, Serbia, Belgrade, November 2009.
34. I. Khemapech, A. Miller, I. Duncan, "Simulating Wireless Sensor Networks", University of St Andrews North Haugh, St Andrews, Scotland.
35. NS-3 overview, <http://www.nsnam.org>, August 2010
36. M. Lacage, "Experimentation with NS-3", INRIA, Trilogy Summer School, August 2009
37. GNS3 (online), <http://www.gns3.net>
38. GNS3 wiki (online), <http://wiki.gns3.net>
39. M. Fuszner, GNS3, Graphical Network Simulator version 1.0, GNS3 tutorial
40. Wireshark project (online), <http://www.wireshark.org/>
41. OPNET Technologies Ltd. (online), <http://www.opnet.com>
42. J. Prokkola, "OPNET – Network Simulator", VTT Technical Research Centre of Finland, Oulu, 2006
43. OMNeT++ (online), <http://www.omnetpp.org>
44. Mobility framework for OMNeT++ (online), <http://mobility-fw.sourceforge.net>
45. GloMoSim (online), <http://pcl.cs.ucla.edu/projects/gloimosim/>
46. QualNet (online), <http://www.scalable-networks.com/products/qualnet/>
47. A. Sobeih, W. Chen, J.C. Hou, L. Kung, N. Li, H. Lim, H. Tyan, H. Zhang, "J-Sim: A simulation and emulation environment for wireless sensor networks", *ANSS 2005*, San Diego, April 2005
48. JiST/SWANS (online), <http://jist.ece.cornell.edu>
49. Ptolemy II (online), <http://ptolemy.berkeley.edu/ptolemyII>
50. P. Baldwin, S. Kohli, E.A. Lee, X. Liu, Y. Zhao, "VisualSense: Visual modeling for wireless and sensor network systems", UCB ERL Memorandum, April 2004.
51. P. Baldwin, S. Kohli, E. A. Lee, X. Liu, Y. Zhao, "Modeling of Sensor Nets in Ptolemy II", Berkeley, CA, April 2004.
52. P. Levis, N. Lee, "TOSSIM: A Simulator for TinyOS Networks", Berkeley, California, 2003
53. Castalia (online), <http://castalia.npc.nicta.com.au/>

54. L. Girod, J. Elson, A. Cerpa, T. Stathopoulos, N. Ramanathan, D. Estrin, "EmStar: a Software Environment for Developing and Deploying Wireless Sensor Networks", CENS Systems Lab, 2007
55. ATEMU (online), <http://www.hynet.umd.edu/research/atemu>
56. J. Polley, D. Blazakis, J. McGee, D. Rusk, J.S. Baras, M. Karir, "ATEMU: A Fine-grained Sensor Network Simulator", 1st IEEE Int. Conf. Sensor and Ad Hoc Communication Networks SECON 04, Santa Clara, 2004
57. G. Chen, J. Branch, M.J. Pflug, L. Zhu, B.k. Szymanski, "SENSE: A Wireless Sensor Network Simulator", Springer, New York, 2004, pages 249-267
58. J.A. Peterson, "SENS: A Simple, Extensible Sensor Network Simulator for Exploring Approximate Aggregation Techniques", Boston University, 2005
59. S. Sundresh, W. Kim, G. Agha, "SENS: A Sensor, Environment and Network Simulator", University of Illinois at Urbana-Champaign
60. JProwler (online), <http://w3.isis.vanderbilt.edu/projects/nest/jproowler/index.html>
61. O. Landsiedel, K. Wehrle, B.L. Titzer, J. Palsberg, "Enabling Detailed Modeling and Analysis of Sensor Networks", SensorNetze, K.G. Saur Verlag, Munchen, 2005.
62. Avrora (online), <http://compilers.cs.ucla.edu/avrora/>
63. B.L. Titzer, D. K. Lee, J. Palsberg, "Avrora: Scalable Sensor Network Simulation with Precise Timing", IPSN 05 Proceedings of the 4th international symposium on Information processing in sensor networks, 2005.
64. F. Osterlind, A. Dunkels, J. Eriksson, N. Finne, T. Voigt, "Cross-Level Sensor Network Simulation with COOJA", First IEEE International Workshop on Practical Issues in Building Sensor Network Applications, Tampa, Florida, 2006
65. COOJA/Contiki (online), <http://www.sics.se/contiki>
66. SwarmNet project (online), <http://shawn.sourceforge.net/>
67. A. Kroeller, D. Pfisterer, C. Buschmann, S. Fekete, S. Fischer, "Shawn: A new approach to simulating wireless sensor networks", Arxiv preprint cs/0502003, 2005
68. A. Sood, "Network Design by using OPNET IT Guru Academic Edition Software", Rivier Academic Journal, Volume 3, Number 1, Spring 2007
69. Consensus home page (online), <http://www.consensus.tudelft.nl/software.html>
70. Mixim (online), <http://mixim.sourceforge.net/>
71. A. Kopke and coauthors, "Simulating Wireless and Mobile Networks in OMNeT++ The MiXiM Vision, OMNeT++ Workshop, Marseille, France, 2008.
72. TinyOS (online), <http://www.tinyos.net>
73. MICA2 and MICAZ (online), <http://www.xbow.com>
74. TMote Sky (online), <http://www.bandwavetech.com/download/tmote-sky-datasheet.pdf>
75. ESB/2 (online), <http://www.sics.se/~adam/contiki/contiki-2.0-doc/a00435.html>
76. H. Sundani, H. Li, V.K. Devabhaktuni, M. Alam, P. Bhattacharya, "Wireless Sensor Network Simulators, A Survey and Comparisons", International Journal of Computer Networks (IJCN), Volume 02, Issue 05, 2010.
77. T.S. Rappaport, "Wireless Communications, principles and practice", Second Edition, Prentice Hall, 2002
78. D. Wagner, R. Wattenhofer, "Algorithms for Sensor and Ad Hoc Networks", Advanced Lectures, Springer, 2007
79. G.V. Merret, N. M. White, N. R. Harris, B. M. Al-Hashimi, "Energy-Aware Simulation for Wireless Sensor Networks", Sixt Annual IEEE Communications Society Conference on Sensor, Mesh and Ad Hoc Communications and Networks (SECON 2009), Rome, Italy, June 2009
80. J.C. Giacomini, L. H. A. Correia, T. Heimfarth, G.M. Pereira, V. F. Silva, J. L. P. de Santana, "Radio Channel Model of Wireless Sensor Networks Operating in 2.4 GHz ISM Band, INFOCOMP (UFLA), v.9. pp, 98-108, 2010
81. A. Savvides, C. Han, M. B. Strivastava, "Dynamic fine-grained localization in Ad-Hoc networks of sensors", Proceedings of the Seventh annual international conference on Mobile computing and networking (MobiCom 01), pp. 166-179, New York, USA, 2001.
82. L. Krishnamurthy, R. Adler, P. Buonadonna, J. Chhabra, M. Flanigan, N. Kushalnagar, L. Nachman, M. Yarvis, "Design and Deployment of Industrial Sensor Networks: Experiences from a Semiconductor Plant and the North Sea", SenSys 05, November 2005, San Diego, USA
83. N. Sarkar, S.A. Halim, "A Review of Simulation of Telecommunication Networks: Simulators, Classification, Comparison, Methodologies, and Recommendations", Cyber Journals: Multidisciplinary Journals in Science and Technology, Journal of Selected Areas in Telecommunications (JSAT), March 2011.
84. H. Chen, "Comparative Study of C, C++, C# and Java Programming Languages", Vaasan Ammattikorke Akoulu, University of Applied Sciences, Information Technology, 2010.